

From Serialized Reasoning Traces to Interactive Reasoning Graphs

Structuring Kimi K3 reasoning for visual exploration, verification, and human intervention

Nathan Conklin

nathan.conklin@vt.edu

Department of Computer Science,
Virginia Tech

Abstract

Large reasoning models can expose extended reasoning traces, but the exposed artifact is still text. Kimi K3, for example, returns a `reasoning_content` field and requires that preserved reasoning history be carried forward in multi-turn use. The interface therefore has access to a serialized record of the model's reasoning, not an explicit graph of goals, premises, checks, assumptions, and conclusions. That distinction creates a practical HCI problem: long textual traces may contain useful structure, but users must reconstruct that structure mentally before they can inspect or intervene in it.

This whitepaper proposes an architecture that converts serialized Kimi K3 reasoning into a structured directed acyclic graph (DAG), maintains one graph for each response or "knowledge card," links those graphs at the conversation level, and renders the result as an interactive visual workspace. The extraction layer is grounded in ReasoningFlow, which models reasoning traces as fine-grained DAGs with typed nodes and edges [1]. The interaction layer is positioned relative to ReTrace [2] and ReasonDiag [3], which demonstrate that structured visualizations can improve comprehension and help users diagnose reasoning errors. The proposed contribution is the integration of these ideas into an incremental, editable interface in which a person can explore, annotate, compare, and eventually modify a model's reconstructed reasoning structure while preserving the original trace as provenance.

1. Motivation

Reasoning interfaces usually expose one of two artifacts: the final answer or a long textual reasoning trace. The final answer hides intermediate structure. The raw trace exposes more information, but it remains difficult to inspect because the user must infer dependencies among steps, notice when the model changes direction, distinguish assumptions from conclusions, and remember which earlier statements support later ones.

Kimi K3 makes this problem concrete. Its API documentation describes K3 as a reasoning model that always runs with thinking enabled and returns `reasoning_content` alongside the assistant response [4]. In a long conversation, an application can preserve that field for history and replay. The API does not, however, return a ready-made decision graph or editable network of reasoning objects. The application therefore receives serialized reasoning that must be interpreted before it can support graph-based interaction.

The design discussed here treats the serialized trace as source material rather than the final UI. The system preserves the original text, derives a structured graph from it, and makes the graph the primary interaction object. This separation is important. The raw trace remains the provenance record; the graph is a designed abstraction intended for human comprehension and intervention.

A useful mental model is a flattened graph. The reasoning trace may contain local evidence of branching, verification, self-reflection, planning, and correction even though those relationships appear in sequence. Terms such as "therefore," "however," "consider," "verify," and "assume" can signal local structure, but lexical rules alone are not sufficient. A practical

system needs a staged extraction process that segments the trace, classifies functional roles, detects dependencies, and records uncertainty about the reconstruction.

2. Terminology: DAG, Streaming, and Knowledge Cards

A directed acyclic graph has directed edges and no cycles. In the ReasoningFlow formulation, edges point from earlier reasoning steps to later steps, which preserves the left-to-right causal ordering of autoregressive generation while allowing one node to depend on several earlier nodes [1]. The graph can represent branching, convergence, verification, and alternative paths without forcing the trace into a simple list.

The term "asynchronous directed graph" appeared during the source discussion but was later corrected. The relevant graph type is a directed acyclic graph. Asynchrony describes the application behavior when the graph is constructed or updated while tokens and response objects arrive; it is not a property of the graph definition.

The discussion also used "knowledge card" as a UI term for an assistant response unit. This whitepaper keeps that term because it is useful for architecture, while treating it as an application concept rather than a Kimi API term. Each knowledge card contains the final answer plus the reasoning trace associated with that response. The proposed system builds a local DAG for each card, then links those local DAGs through a higher-level conversation graph.

3. What Kimi K3 Provides

Official Kimi documentation currently states that K3 always has thinking enabled and returns `reasoning_content`; thinking effort is controlled through the top-level `reasoning_effort` parameter [4]. For multi-turn interactions, Kimi instructs developers to preserve the complete assistant message, including `reasoning_content` and tool calls, when sending prior turns back to the model [5]. That behavior is well suited to a reasoning visualization prototype because the application can retain both the visible answer and the associated trace.

The following object is illustrative rather than a normative copy of the complete Kimi API schema. It shows the fields that matter to the proposed visualization pipeline.

```
{
  "role": "assistant",
  "reasoning_content": "Serialized reasoning trace...",
  "content": "Final answer shown to the user",
  "tool_calls": []
}
```

The visualization system should preserve the original assistant object before it performs any post-processing. A reconstructed graph is an interpretation of the trace; it should never overwrite the source. This creates a clean provenance chain:

```
Kimi assistant message
-> immutable source trace
-> graph extraction pipeline
-> JSON reasoning graph
-> interactive visualization
-> user annotations and edits
```

[TODO: confirm the exact response object and field mapping produced by the specific Kimi K3 endpoint or OpenAI-compatible gateway used in the prototype. Official Kimi documentation returns `reasoning_content`, but intermediary providers may map or omit fields.]

4. ReasoningFlow as the Structural Foundation

ReasoningFlow is the closest structural foundation identified in the source discussion. Lee et al. model long reasoning traces as fine-grained DAGs and define eight node types and fourteen edge types [1]. Their motivation closely matches the reconstruction problem: reasoning models backtrack, verify, reflect, and correct themselves, so a flat sequence can obscure which steps influence later conclusions.

ReasoningFlow uses contiguous, non-overlapping spans as nodes. A sentence normally becomes one node, but a sentence can be divided when clauses serve different functions. The schema includes core roles for reasoning, planning, and reflection, plus special cases such as facts, restatements, assumptions, examples, and conclusions [1]. The edge vocabulary captures several families of relations, including derivation, plan execution, reflection, and validation.

The paper's automatic annotation pipeline uses three stages [1]:

1. **Node segmentation.** Split the raw trace into elementary reasoning units.
2. **Node classification.** Assign a functional role to each unit.
3. **Edge detection and classification.** Identify incoming dependencies for each node and label the relationship.

The authors use Gemini-3.1-Flash for node annotation and Gemini-3-Pro for edge annotation after evaluating those models against manually annotated examples [1]. They also manually review a subset of automatically generated graphs. Their dataset contains 1,260 automatically annotated traces spanning math, science, and argumentation tasks from five models: Qwen2.5-32B-Instruct, QwQ-32B, DeepSeek-V3, DeepSeek-R1, and GPT-oss-120B [1]. The code and dataset are public [6][7]. As of August 2026, the work is listed as accepted to the EMNLP 2026 Main Conference [8].

This work reduces the structural risk of the proposed prototype. The first research problem does not need to be "invent a graph representation for reasoning from nothing." A stronger starting point is to adopt or adapt an existing ontology, validate it on Kimi K3 traces, and concentrate original effort on interaction, incremental reconstruction, and cross-turn reasoning.

4.1 Node Quality as a Separate Layer

ReasoningFlow also annotates node quality. For reasoning tasks, the authors use an LLM judge to label logical validity; for the argumentation task, they connect reasoning nodes to a human-rated argument-quality resource [1]. This distinction is useful for interface design because graph topology and graph quality are different properties. One layer explains how steps relate. Another layer estimates whether a step is sound.

An interactive system should keep these layers separate. A node can be central to the reasoning path while still being low quality; a node can also be erroneous but disconnected from the final conclusion. The interface can therefore encode at least three attributes independently: functional node type, structural dependency, and quality or confidence.

5. Related Visualization Work

ReasoningFlow addresses structural extraction, but the proposed work also needs an HCI foundation. Two recent visualization systems provide that context.

5.1 ReTrace

ReTrace structures and visualizes textual reasoning traces using a validated reasoning taxonomy [2]. The authors evaluate two interactive visualizations in a controlled human-subject study. Participants understood the model's reasoning more accurately and reported less perceived effort than with a raw text baseline [2]. The result supports a central design assumption of this whitepaper: transforming a long trace into an interactive visual representation can improve comprehension rather than merely decorate the text.

ReTrace is accepted in *Computer Graphics Forum* with DOI 10.1111/cgf.70452 [9]. Its strongest connection to the proposed work is interaction and comprehension. ReasoningFlow offers a rich DAG representation; ReTrace provides evidence that structured interactive views can make reasoning traces easier to understand.

5.2 ReasonDiag: When the Chain Breaks

Chen et al. introduce ReasonDiag in *When the Chain Breaks: Interactive Diagnosis of LLM Chain-of-Thought Reasoning Errors* [3]. The system combines an error-detection pipeline with an integrated arc diagram and a hierarchical node-link diagram. The visualizations show reasoning-step distributions, error propagation, high-level reasoning flow, and premise dependencies. The authors evaluate the system through technical evaluation, case studies, and interviews with 16 participants [3].

ReasonDiag is particularly relevant to a later prototype stage because it treats error diagnosis as an interaction problem. Once the proposed system has a stable graph, similar techniques can help users locate suspect nodes, see which downstream conclusions depend on them, and understand whether an error propagated or was abandoned.

5.3 Position of the Proposed Work

The three related efforts form a useful division of labor.

Work	Primary contribution	Relevance to this project
ReasoningFlow [1]	Fine-grained DAG representation and extraction of reasoning structure	Structural schema and extraction algorithm
ReTrace [2]	Interactive visualization of structured reasoning traces	Comprehension-oriented interaction patterns and empirical motivation
ReasonDiag [3]	Interactive diagnosis of step-level reasoning errors	Error propagation, premise dependencies, diagnostic workflows
Proposed Kimi K3 interface	Incremental, editable, per-card and cross-card reasoning graph	Human intervention, streaming updates, conversation-scale structure

The proposed contribution sits at their intersection. It uses a graph representation similar to ReasoningFlow and turns that graph into a persistent UI object that users can inspect and eventually manipulate while a conversation develops.

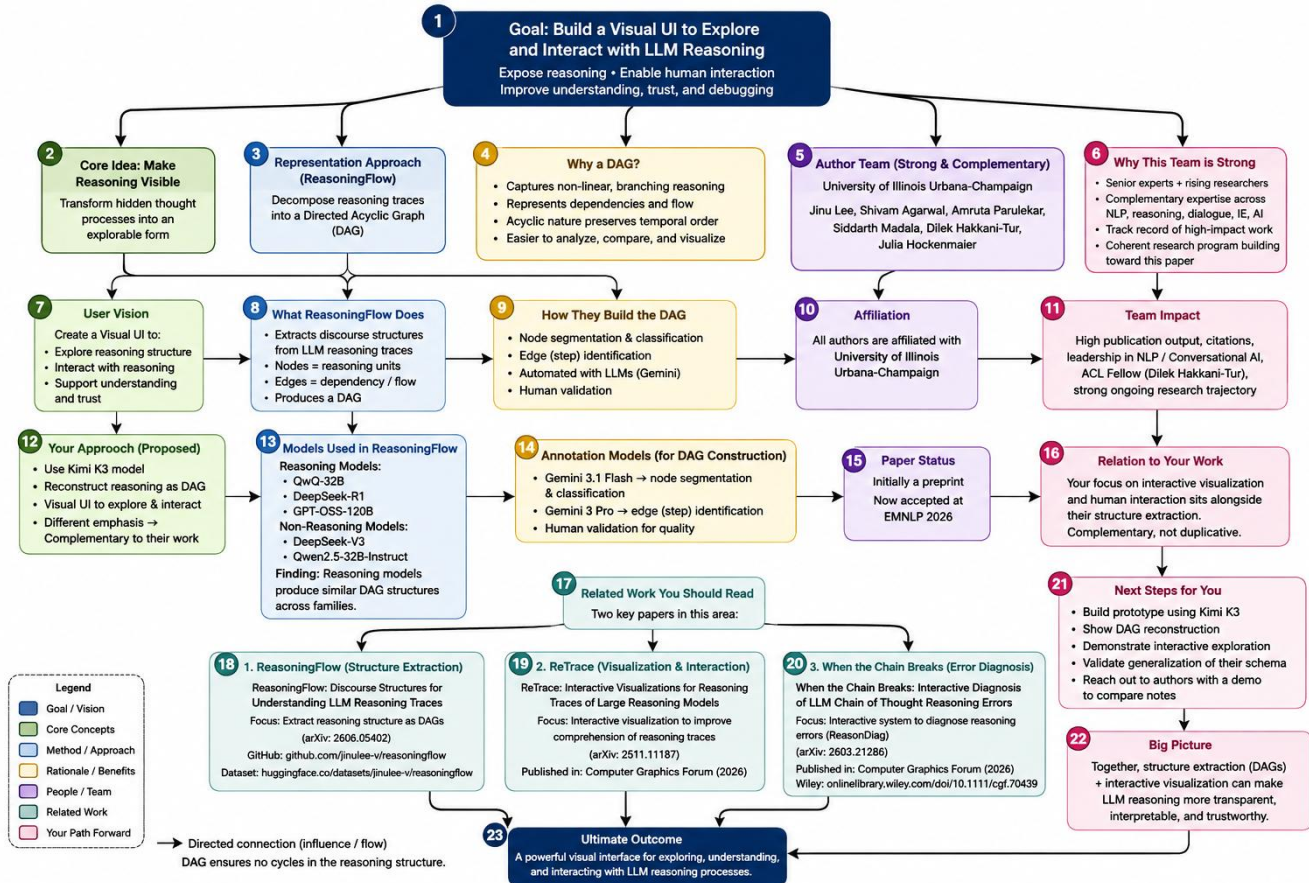


Figure 1: Research landscape and proposed interactive reasoning direction

6. Proposed Graph Construction Pipeline

A practical implementation should separate extraction into pluggable stages. This follows the structure of ReasoningFlow while allowing the prototype to start with simpler methods and replace them later.

6.1 Stage 0: Preserve the Source

Store the complete assistant message and assign stable identifiers to the conversation, knowledge card, and trace. Preserve exact text offsets. Every graph node should point back to its source span so a user can inspect the original reasoning that produced it.

6.2 Stage 1: Segment the Trace

Split the reasoning trace into candidate units. A first implementation can use sentence and clause boundaries, then ask an LLM to split clauses that contain distinct roles. Segmentation should preserve character offsets and token positions where possible.

A node record can start with a small schema:

```
{
  "id": "kc12-n17",
  "card_id": "kc12",

```

```
"source_start": 842,
"source_end": 931,
"text": "I should verify the previous assumption.",
"type": "planning",
"quality": null
}
```

6.3 Stage 2: Classify Node Function

Assign each node a functional type. The prototype should begin by reusing ReasoningFlow labels rather than inventing a large application-specific ontology immediately. Additional UI-specific attributes can be layered on later, such as user-created notes, confidence, hidden/collapsed state, or provenance tags.

6.4 Stage 3: Detect and Classify Edges

For each node, identify which earlier nodes it depends on and label the relation. Asking for incoming edges one node at a time limits output size and mirrors the ReasoningFlow annotation strategy [1]. The result should remain acyclic by construction because an edge can only point from an earlier source span to a later one.

```
{
  "from": "kc12-n11",
  "to": "kc12-n17",
  "relation": "plan_verify",
  "confidence": 0.82
}
```

The confidence field above is a proposed application attribute, not a field asserted to exist in ReasoningFlow. It records confidence in the extraction so the UI can distinguish a confident dependency from a speculative one.

6.5 Stage 4: Quality and Verification

Run optional node-level quality checks after graph construction. These can include logical validity, source support, contradiction checks, or task-specific validation. The graph should display quality separately from structural type. A red node should mean "this step may be wrong," not "this node type is an error."

6.6 Stage 5: Graph Validation

Before rendering, validate machine-readable invariants:

- all node identifiers are unique;
- every edge references existing nodes;
- every edge points forward within the local trace unless it is explicitly a cross-card reference;
- the local card graph is acyclic;
- every node retains a source span;
- every inferred label records the model or rule that produced it.

[TODO: inspect the current ReasoningFlow repository and released dataset before implementation to confirm the exact JSON field names, serialization format, and license requirements. The source discussion identified JSON as the primary artifact, but the production prototype should use the repository's current schema rather than a reconstructed approximation.]

7. One Graph Per Knowledge Card, One Graph Across the Conversation

The source discussion identified an important architectural choice: do not begin with one monolithic graph for an entire chat. Build a local graph for each knowledge card, then construct a higher-level graph across cards.

This two-level representation has several advantages. Local extraction is bounded by one response, so parsing and validation remain manageable. A card can be rendered as soon as its reasoning completes. Failures are isolated; one poor extraction does not corrupt the entire conversation. The user can also collapse old cards while keeping a summary of their relationships.

The conversation graph then links card-level concepts. A cross-card edge might represent reuse of an earlier assumption, continuation of a plan, contradiction of a previous conclusion, or reference to an earlier piece of evidence. These cross-card links require a different policy from local edges because semantic identity is ambiguous. The system must decide whether two statements are the same concept, related concepts, or distinct historical events.

The safest initial design is to preserve history and add links rather than merge nodes aggressively. If two cards discuss the same assumption, keep both source nodes and connect them with a relation such as `restates`, `revises`, or `refers_to`. Later research can test whether semantic deduplication improves or harms comprehension.

[TODO: define the cross-card identity policy, including when semantically similar nodes remain separate historical events and when the UI presents them as one persistent concept.]

8. Client and Server Architecture

The prototype discussed in the source uses React, Node.js, and FastAPI. The graph layer fits that stack well, but the responsibilities should be explicit.

```

Browser: React + TypeScript
- conversation UI
- per-card graph state
- graph validation
- visual layout and interaction
- user annotations / edits
- cross-card linking UI
    |
    | JSON / streaming events
    v
FastAPI service
- Kimi API credential boundary
- request forwarding / streaming
- durable transcript storage
- optional LLM annotation calls
    |
    v
Kimi K3 and optional annotation model

```

The browser should own the interactive graph state because that state changes rapidly with user actions. The browser does not need a direct port of the ReasoningFlow Python implementation. A TypeScript graph model can implement the same schema and algorithmic stages. The FastAPI service should retain API credentials and can optionally perform model-assisted segmentation or classification when those calls require protected credentials or heavier processing.

This architecture keeps the parser swappable. An early prototype can combine sentence splitting with LLM classification. A later version can call a dedicated annotation model, reuse ReasoningFlow code on the server, or run a local open-weight model. The React UI should consume the same graph contract regardless of which extractor produced it.

For rendering, React Flow and Cytoscape are both reasonable candidates. React Flow is well aligned with editable node-based applications; Cytoscape provides mature graph analysis and layout support. The graph contract should remain independent of the rendering library so the research does not become tied to one component toolkit.

9. Interaction Design

The research opportunity begins after the graph is reconstructed. A static DAG can help a user see structure, but an interactive reasoning interface should support direct exploration and intervention.

Candidate interactions include:

- expand or collapse a branch;
- select a node and reveal the exact source text;
- highlight all ancestors that support a conclusion;
- highlight all descendants affected by an assumption;
- filter by node type, quality, or extraction confidence;
- annotate a node with a user note;
- mark a node as accepted, disputed, or unresolved;
- compare an original node with a later revision;
- edit an assumption and request a new continuation from that point;
- replay the reasoning path that led to a selected conclusion.

The first prototype should distinguish graph editing from model execution. A user may change a label or add an annotation without asking Kimi to regenerate anything. A later "continue from here" action can create a new branch that records the user intervention as provenance. This keeps the interface inspectable and makes the difference between editing the visualization and changing the model's future reasoning explicit.

10. Worked Visual Example: Actor Connectedness

The actor-connectedness example from the source discussion provides a simple visual grammar for a DAG without exposing long reasoning text. Julia Roberts is the start node, Robert Downey Jr. is the end node, and intermediate actor names form multiple directed paths. The example demonstrates branching and convergence while keeping each node small enough to scan.

Directed Acyclic Graph of Actor Connections: Julia Roberts to Robert Downey Jr.

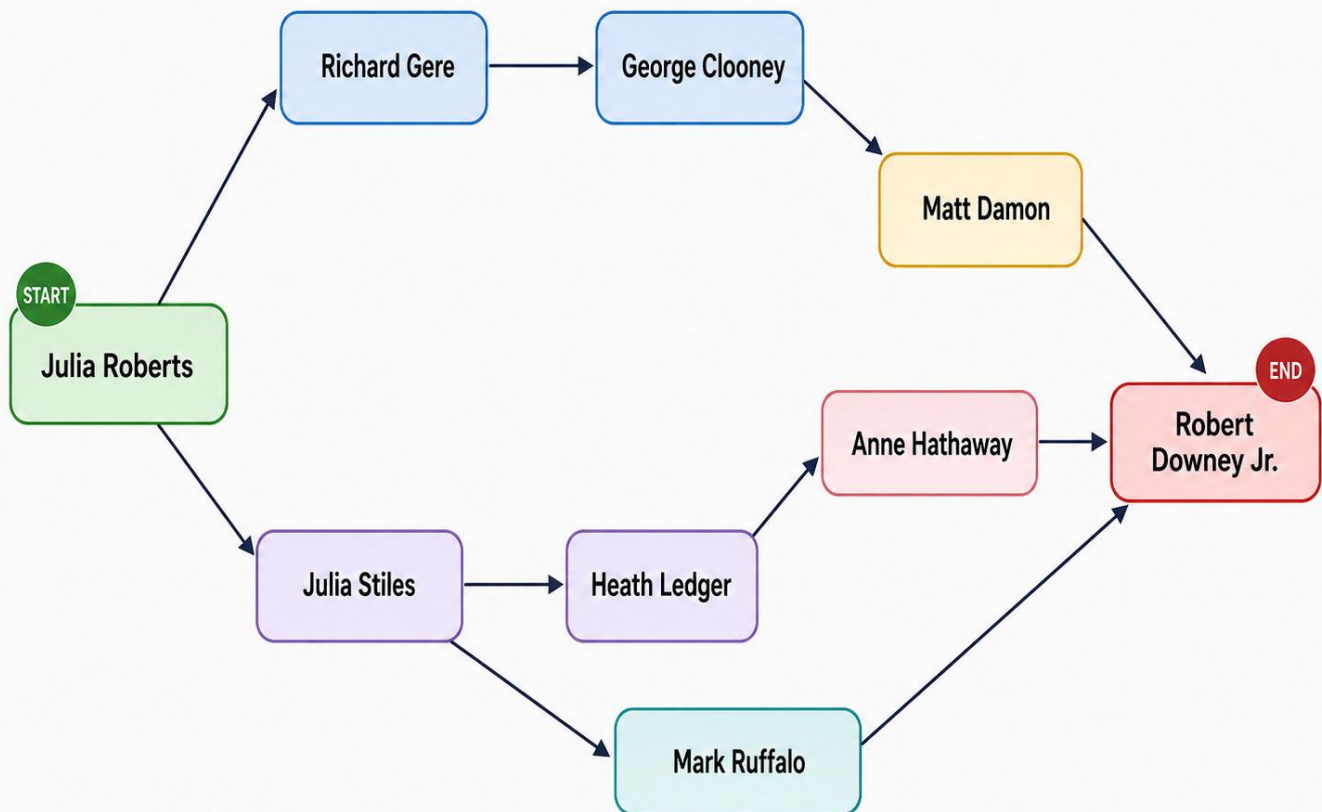


Figure 2: Illustrative actor-connectedness DAG used to demonstrate branching and convergence

The figure should be interpreted as an interface example rather than a verified filmography dataset. Its value is structural. A reasoning interface can use the same visual grammar while replacing actor names with reasoning units, evidence, assumptions, verification steps, and conclusions. Start and end emphasis can become goal and answer emphasis; branching can represent alternative hypotheses or attempts; convergence can show multiple premises supporting one conclusion.

The example also reveals an important layout principle. A useful reasoning graph should not render every relationship with equal weight. Decision points, corrections, and convergence points deserve more visual emphasis than routine intermediate steps. Progressive disclosure can keep large graphs readable by collapsing low-value chains until the user asks for detail.

11. Research Questions and Evaluation Path

The source discussion moved the research framing from "can a graph be recovered?" toward "how can a recovered graph become an interactive reasoning instrument?" That framing suggests four candidate research questions.

RQ1: Structural Fidelity

How accurately can a ReasoningFlow-derived schema reconstruct Kimi K3 reasoning traces? A validation study can compare automated graph extraction with human annotation on a sample of Kimi traces. Metrics can include node segmentation agreement, node classification F1, edge detection F1, and edge-label agreement.

RQ2: Comprehension

Does an interactive graph improve a user's understanding of long reasoning traces compared with raw text? ReTrace provides precedent for evaluating accuracy and perceived effort [2]. A Kimi-specific study can test whether users answer dependency, correction, and conclusion questions more accurately with the graph.

RQ3: Verification and Intervention

Can users find weak assumptions, trace their downstream effects, and make targeted interventions more effectively with the graph? ReasonDiag provides a useful diagnostic comparison [3], but the proposed interface adds direct manipulation and conversation continuation.

RQ4: Conversation-Scale Reasoning

Does a two-level graph, local DAGs per knowledge card plus cross-card links, help users manage long conversations better than a single monolithic trace view? This question is specific to the proposed architecture and may become a distinct HCI contribution.

[TODO: define the first empirical study, participant population, baseline interfaces, tasks, and success metrics before making claims about user benefit.]

12. Implementation Sequence

A staged implementation can reduce risk while keeping the prototype usable after each step.

Phase 1: Capture and Persistence

Capture the complete Kimi assistant response, including `reasoning_content`, and store it with stable card identifiers. Display the raw trace on demand. This proves that the application receives and retains the necessary data.

Phase 2: Static Per-Card DAG

Implement segmentation, node classification, edge detection, graph validation, and JSON serialization for one knowledge card. Render the result after the response completes. At this stage, the graph may be post-hoc rather than streamed.

Phase 3: Interactive Per-Card DAG

Add source-span inspection, branch collapse, filtering, annotations, node quality overlays, and ancestor/descendant highlighting. Record all user actions separately from the machine-generated graph.

Phase 4: Conversation Graph

Add cross-card links and navigation. Preserve historical nodes by default; introduce semantic merge only after testing its effect on comprehension.

Phase 5: Human Intervention

Allow a user to revise an assumption, choose an alternative branch, or request a continuation from a selected node. The system should create a new branch rather than rewriting history. Provenance should show the original model reasoning, the human intervention, and the resulting continuation.

Phase 6: Evaluation

Compare the interface against raw reasoning text and a simpler linear trace visualization. Measure structural understanding, task accuracy, time, perceived effort, trust calibration, and intervention quality where appropriate.

13. Design Constraints and Risks

13.1 Reconstructed Structure Is an Interpretation

A DAG derived from text can look authoritative even when the extraction model guessed. The UI should expose extraction confidence and preserve links to source spans. Users need a way to see which relationships are directly indicated by the text and which were inferred.

13.2 Reasoning Traces Are Not Guaranteed to Be Faithful Explanations

A visible reasoning trace can be useful for debugging and interaction without proving that it is a complete causal account of the model's internal computation. ReasoningFlow itself distinguishes language-level discourse structure from mechanistic causal dependency and reports that the two do not align cleanly [1]. The UI should therefore describe the graph as reconstructed reasoning discourse, not as a literal map of model internals.

13.3 Large Graphs Can Recreate the Original Cognitive Burden

A graph with hundreds of uniformly visible nodes may be harder to use than the raw text. The UI needs progressive disclosure, semantic zoom, filtering, and emphasis on important nodes. The research should evaluate whether the chosen abstraction reduces cognitive load rather than assuming that a graph is inherently clearer.

13.4 Cross-Card Identity Is Difficult

Long conversations repeatedly restate, revise, and rename concepts. Automatic node merging can erase useful history. The initial system should favor explicit cross-links and user-confirmed merges.

13.5 Provider and API Behavior Can Change

The prototype should isolate provider-specific parsing behind a small adapter. Kimi K3 currently documents `reasoning_content`, but field names, compatibility behavior, and available reasoning modes may change [4][5]. The graph pipeline should accept a normalized trace object rather than depend directly on one wire format.

14. Conclusion

The source discussion began with a practical question: Kimi K3 exposes reasoning, but the reasoning still arrives as serialized text. Recent research provides enough foundation to avoid treating graph reconstruction as a blank-slate problem. ReasoningFlow supplies a validated DAG schema and annotation process; ReTrace and ReasonDiag show that structured interactive views can improve comprehension and diagnosis.

The proposed system combines those threads into a conversation-scale interface. It preserves Kimi's original reasoning trace, reconstructs a local DAG for each knowledge card, links those graphs across the chat, and treats the resulting structure as an interactive object rather than a static analysis artifact. The strongest research opportunity is the design and evaluation of a visual workspace that lets a person understand, verify, and intervene in reconstructed machine reasoning while preserving provenance and uncertainty.

References

- [1] Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala, Dilek Hakkani-Tur, and Julia Hockenmaier. 2026. *ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces*. arXiv:2606.05402; accepted to the EMNLP 2026 Main Conference. <https://arxiv.org/abs/2606.05402>
- [2] Ludwig Felder, Jacob Miller, Markus Wallinger, Stephen G. Kobourov, and Chunyang Chen. 2026. *ReTrace: Interactive Visualizations for Reasoning Traces of Large Reasoning Models*. Computer Graphics Forum. DOI: 10.1111/cgf.70452. <https://arxiv.org/abs/2511.11187>
- [3] Shiwei Chen, Niruthikka Sritharan, Xiaolin Wen, Chenxi Zhang, Xingbo Wang, and Yong Wang. 2026. *When the Chain Breaks: Interactive Diagnosis of LLM Chain-of-Thought Reasoning Errors*. Computer Graphics Forum, e70439. DOI: 10.1111/cgf.70439. <https://doi.org/10.1111/cgf.70439>
- [4] Kimi. 2026. *Model selection: kimi-k3*. Kimi API Help Center. <https://www.kimi.ai/help/kimi-api/api-model-selection>
- [5] Moonshot AI. 2026. *Kimi K3 model usage and preserved thinking history*. Kimi K3 model documentation. <https://huggingface.co/moonshotai/Kimi-K3>
- [6] Jinu Lee et al. 2026. *ReasoningFlow source code*. GitHub. <https://github.com/jinulee-v/reasoningflow>
- [7] Jinu Lee et al. 2026. *ReasoningFlow dataset*. Hugging Face Datasets. <https://huggingface.co/datasets/jinulee-v/reasoningflow>
- [8] Conversational AI Lab, University of Illinois Urbana-Champaign. 2026. *Publications: ReasoningFlow accepted at EMNLP 2026*. <https://uiuc-conversational-ai-lab.github.io/publications/>
- [9] Technical University of Munich. 2026. *ReTrace: Interactive Visualizations for Reasoning Traces of Large Reasoning Models*. Research portal; accepted/in press in Computer Graphics Forum. <https://doi.org/10.1111/cgf.70452>